

✓ Transfer Learning for Classification

Please save a copy of the notebook in your local drive! Otherwise you will lose your progress.

✓ Introduction & a Bit of History

Over the course of CS189, you have trained machine learning models for specific applications, whether it is classification or regression. Although this has been effective, modern machine learning researchers use a considerably different pipeline for training models.

What people have been doing instead is to train a massive model for a very broad objective, and then finetune the model on a smaller dataset. This approach has given us a lot of great-performing models and agents, the most notable being that of [GPT](#) models (there are also more models that does the same job, BERT, DINO, masked autoencoders are the ones that immediately came to mind, but unfortunately there are no equivalent in robotics), as they relied on internet scale pretraining (with GPT specifically, people use what's called next-token prediction as the objective). You don't need to know exactly how this works in this class, but this is drastically different than what we have done by fitting a model to exactly one dataset.

Of course, we do not have the computational resources to retrain any GPT3+ models, but we can try a smaller problem with transfer learning! Here is a little story behind the origin of transfer learning:

Back in 2013, neural-net based image recognition methods such as AlexNet has already swept through the ML community by storm, and people are trying to tackle the next problem in computer vision, which is object segmentation (using bounding boxes). The problem is that the corresponding dataset, [Pascal-VOC](#), is very small for neural networks (only 1500 training images and 20,000 data entries).

So what did people do? Researchers have found out is that instead of training a neural network from scratch, using weights from networks trained on [ImageNet classification](#) as the starting point actually helped to achieve very good tracking accuracy on these tasks (if you are so inclined, feel free to read [R-CNN](#) to see what they have done)! In fact, this method worked so well that it not only inspired the idea of "pretraining" across the machine learning community, but also sparked a [funny bet](#) between two UC Berkeley professors.

In previous semesters, we actually didn't want you to do transfer learning for CIFAR-10, but with how much the machine learning world has changed, we are changing the focus specifically on transfer learning to get you more familiar with these paradigms and hope you can have successes working with these large models!

✓ Transfer Learning for CIFAR-10

In this question, we will explore how to effectively use transfer learning to learn a new task with a *pretrained* network. In essence, the idea of pretraining is that a network has been trained on another objective, and that the weights are something more desirable than random initialization. Transfer learning takes advantage of that and aims to make training faster compared to that of training from scratch.

```
# utils for data loading
```

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import datasets
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
from torchvision import models

import time
import numpy as np
import pandas as pd

import os
import time

import random
from tqdm.notebook import tqdm

import matplotlib.pyplot as plt

if torch.cuda.is_available():
    torch.backends.cudnn.deterministic = True
```

```
%reload_ext autoreload
%autoreload 2
%matplotlib inline
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
device
```

```
↗ 'cuda'
```

```
seed = 189
np.random.seed(seed)
torch.manual_seed(seed)
torch.cuda.manual_seed(seed)
```

First, let's start with making the necessary transformations of our training data. CIFAR-10 is a collection of 60,000 32x32x3 images divided into 10 classes. If you want more information, please refer to [this link](#) by Alex Krizhevsky et al.

However, most of the models readily available are trained on ImageNet, which is 224x224x3. Although one can hack the weights of a neural network and design a new network on top of it, it is much easier to simply scale up the images by a factor of 7 (224/32) and feed the network the magnified images. Let's do that instead!

```
def num_trainable_params(model: nn.Module):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

## transforms to match that of the transforms in ImageNet
transform = transforms.Compose([
    transforms.Resize((224, 224)), # Resize to 224x224 (ImageNet size)
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

## download training dataset
train_dataset = datasets.CIFAR10(root='./data', train=True,
                                  transform=transform, download=True)

test_dataset = datasets.CIFAR10(root='./data', train=False,
                                  transform=transform, download=True)

train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True, num_workers=2)
test_loader = DataLoader(test_dataset, batch_size=128, shuffle=False, num_workers=2)
```

```
↗ 100%|██████████| 170M/170M [00:04<00:00, 41.9MB/s]
```

Let's also visualize some images within CIFAR-10 Dataset!

```
viz_transform = transforms.Compose([
    transforms.Resize((224, 224)), # Resize to 224x224 (height x width)
    transforms.ToTensor(),
])

viz_dataset = datasets.CIFAR10(root='./data', train=True,
                                transform=viz_transform, download=True)

images = [viz_dataset[i][0] for i in range(9)]
plt.imshow(torchvision.utils.make_grid(torch.stack(images), nrow=3, padding=5).numpy().transpose((1, 2, 0)))
```



Now go to `torchvision.models` module, and pick one of the following models that you would like.

- VGG Networks (all VGG networks are fine)
- AlexNet
- ResNet-18, -34, or -50

A more expressive model, such as using vision transformers, will not be allowed (they also have hundreds of millions of parameters, which is not friendly for your colab compute units). All model weights must directly come from `torchvision.models` module.

In order to perform transfer learning, you should do the following steps:

1. Load a pretrained network on another dataset.
2. Freeze most of the layers of the pretrained network and discard a part of the network not relevant to your model.
3. Append a new MLP at the end of the network, and allow gradients to pass through the new MLP (and other non-frozen parts of the network).
4. Train the new network.

Although we would like you to explore transfer learning, here are some rules you must follow:

1. You may not change the model architecture other than the final linear layers.
2. You must freeze all of the weights in the convolutional layers.
3. You may not have more than 3 million trainable parameters.

```
class TransferCIFAR10(nn.Module):
    def __init__(self, *args, **kwargs):
        super().__init__()
        self.base_model = models.resnet34(pretrained=True)

        # use trained model as feature extractor -> freeze all trained parameters
        for param in self.base_model.parameters():
            param.requires_grad = False

        num_features = self.base_model.fc.in_features
        self.base_model.fc = nn.Linear(num_features, 10)

    def forward(self, x):
        return self.base_model(x)
```

If you want to check how many trainable parameters are within your model, run this script here:

```
learning_rate = 0.001

model = TransferCIFAR10().to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=2, gamma=0.5)
criterion = nn.CrossEntropyLoss()
```

```
print("Trainable parameters:", num_trainable_params(model))
```

```

/usr/local/lib/python3.11/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated
warnings.warn(
/usr/local/lib/python3.11/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or
warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/resnet34-b627a593.pth" to /root/.cache/torch/hub/checkpoints/resnet34-b627
100%|██████████| 83.3M/83.3M [00:00<00:00, 118MB/s]
Trainable parameters: 5130

```

```
def train(model, train_loader, optimizer, criterion, device):
```

```

    model.train() # set training mode
    running_loss, correct, total = 0.0, 0.0, 0.0

```

```

    for x, y in tqdm(train_loader, desc="Training", unit="batch", leave=False):
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad()
        outputs = model(x)
        loss = criterion(outputs, y)
        loss.backward()
        optimizer.step()

```

```

        running_loss += loss.item() * x.size(0) # total loss in one batch
        preds = torch.argmax(outputs, dim=1)
        correct += (preds == y).sum().item()
        total += y.size(0)

```

```

    avg_loss = running_loss / len(train_loader.dataset)
    accuracy = correct / total
    return avg_loss, accuracy

```

```
def evaluate(model, test_loader, optimizer, criterion, device):
```

```

    model.eval()
    running_loss, correct, total = 0.0, 0.0, 0.0

```

```

    with torch.no_grad():
        for x, y in tqdm(test_loader, desc="Evaluating", unit="batch", leave=False):
            x, y = x.to(device), y.to(device)
            outputs = model(x)
            loss = criterion(outputs, y)

            running_loss += loss.item() * x.size(0)
            preds = torch.argmax(outputs, dim=1)
            correct += (preds == y).sum().item()
            total += y.size(0)

```

```

    avg_loss = running_loss / len(test_loader.dataset)
    accuracy = correct / total
    return avg_loss, accuracy

```

```
# TRAINING and VALIDATION BLOCK with frozen weights
```

```
frozen_i_val_loss, frozen_i_val_acc = evaluate(model, test_loader, optimizer, criterion, device)
```

```
# initial validation
```

```
frozen_train_losses, frozen_val_losses = [], [frozen_i_val_loss]
```

```
frozen_train_accuracies, frozen_val_accuracies = [], [frozen_i_val_acc]
```

```
frozen_best_val_acc = 0
```

```
epochs = 5
```

```
for epoch in range(epochs):
```

```
    print(f"\nEpoch {epoch + 1} / {epochs} (lr: {scheduler.get_last_lr()[0]:.6f})")
```

```
    frozen_train_loss, frozen_train_acc = train(model, train_loader, optimizer, criterion, device)
```

```
    frozen_val_loss, frozen_val_acc = evaluate(model, test_loader, optimizer, criterion, device)
```

```
    scheduler.step()
```

```
    frozen_train_losses.append(frozen_train_loss)
```

```
    frozen_val_losses.append(frozen_val_loss)
```

```
    frozen_train_accuracies.append(frozen_train_acc)
```

```
    frozen_val_accuracies.append(frozen_val_acc)
```

```

    print(f"Train Loss: {frozen_train_loss:.4f}, Train Acc: {frozen_train_acc:.4f} | "
          f"Val Loss: {frozen_val_loss:.4f}, Val Acc: {frozen_val_acc:.4f}")

```

```
# Save best model
if frozen_val_acc > frozen_best_val_acc:
    frozen_best_val_acc = frozen_val_acc
    torch.save(model.state_dict(), "best_transfer_model_resnet34.pth")
    print("Saved best model.")
```



```
Epoch 1 / 5 (lr: 0.001000)
Train Loss: 0.5933, Train Acc: 0.8013 | Val Loss: 0.5723, Val Acc: 0.8021
Saved best model.
```

```
Epoch 2 / 5 (lr: 0.000500)
Train Loss: 0.5515, Train Acc: 0.8128 | Val Loss: 0.5582, Val Acc: 0.8081
Saved best model.
```

```
Epoch 3 / 5 (lr: 0.000500)
Train Loss: 0.5397, Train Acc: 0.8146 | Val Loss: 0.5509, Val Acc: 0.8095
Saved best model.
```

```
Epoch 4 / 5 (lr: 0.000250)
Train Loss: 0.5255, Train Acc: 0.8209 | Val Loss: 0.5396, Val Acc: 0.8122
Saved best model.
```

```
Epoch 5 / 5 (lr: 0.000250)
Train Loss: 0.5226, Train Acc: 0.8215 | Val Loss: 0.5414, Val Acc: 0.8108
```

```
# Plots for Feature Extraction (Frozen Weights)
```

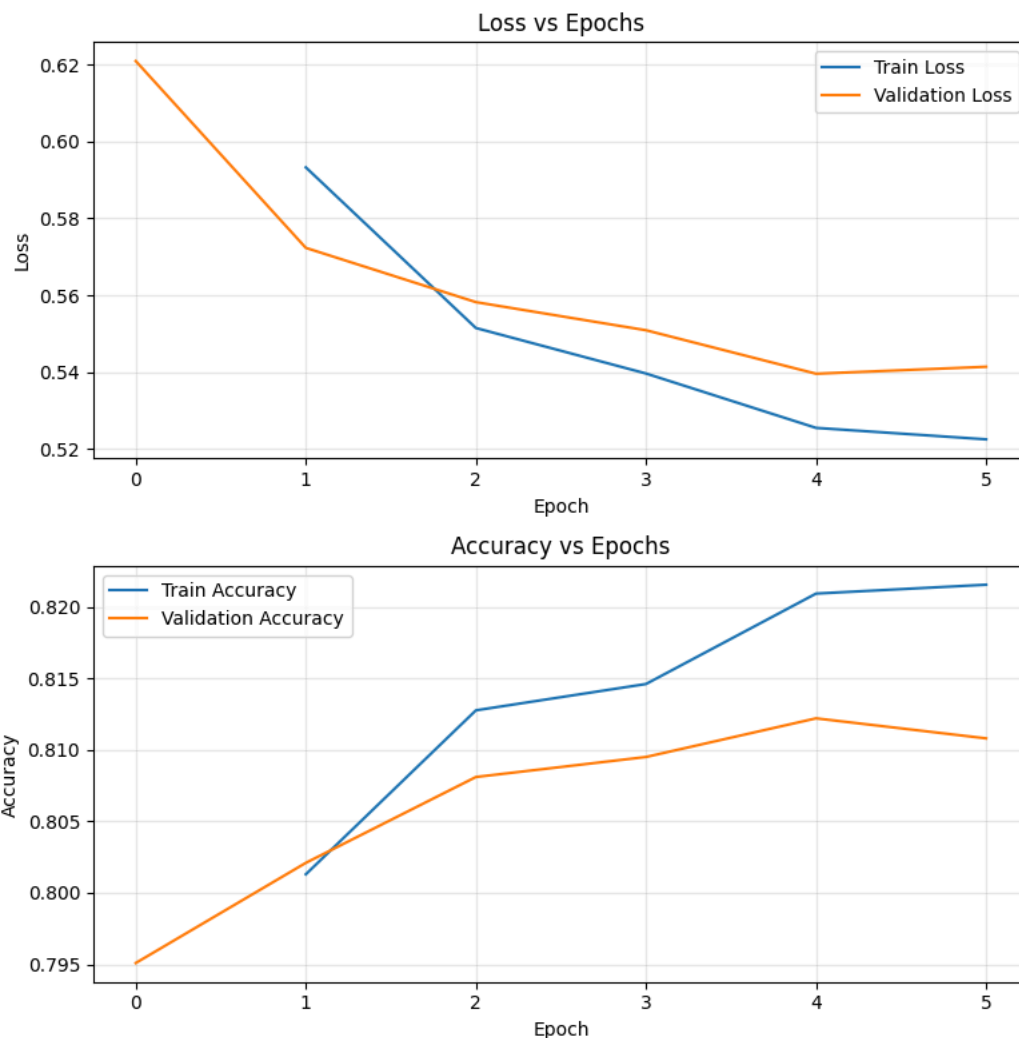
```
epochs_x = range(1, epochs + 1)
epochs_x_val = range(0, epochs + 1)
```

```
fig, (ax1, ax2) = plt.subplots(2, figsize=(8, 8))
```

```
ax1.plot(epochs_x, frozen_train_losses, label="Train Loss")
ax1.plot(epochs_x_val, frozen_val_losses, label="Validation Loss")
ax1.set_xlabel("Epoch")
ax1.set_ylabel("Loss")
ax1.set_title("Loss vs Epochs")
ax1.legend()
ax1.grid(alpha=0.3)
```

```
ax2.plot(epochs_x, frozen_train_accuracies, label="Train Accuracy")
ax2.plot(epochs_x_val, frozen_val_accuracies, label="Validation Accuracy")
ax2.set_xlabel("Epoch")
ax2.set_ylabel("Accuracy")
ax2.set_title("Accuracy vs Epochs")
ax2.legend()
ax2.grid(alpha=0.3)
```

```
fig.suptitle("Feature Extraction ResNet34", fontsize=16)
plt.tight_layout()
plt.show()
```



✓ Fine Tuning

```
class TransferFinetunedCIFAR10(nn.Module):
    def __init__(self, *args, **kwargs):
        super().__init__()
        self.base_model = models.resnet18(pretrained=True)

        num_features = self.base_model.fc.in_features
        self.base_model.fc = nn.Linear(num_features, 10)

    def forward(self, x):
        return self.base_model(x)

finetuned_model = TransferFinetunedCIFAR10().to(device)

learning_rate = 0.00001
optimizer = torch.optim.Adam(finetuned_model.parameters(), lr=learning_rate, weight_decay=1e-4)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=10)
criterion = nn.CrossEntropyLoss()
print("Trainable parameters:", num_trainable_params(finetuned_model))
```



Trainable parameters: 11181642

```
i_val_loss, i_val_acc = evaluate(finetuned_model, test_loader, optimizer, criterion, device)
```

```
train_losses, val_losses = [], [i_val_loss]
train_accuracies, val_accuracies = [], [i_val_acc]
```

epochs = 5

```

best_val_acc = 0

for epoch in range(epochs):
    print(f"\nEpoch {epoch + 1} / {epochs} (lr: {scheduler.get_last_lr()[0]:.6f})")

    train_loss, train_acc = train(finetuned_model, train_loader, optimizer, criterion, device)
    val_loss, val_acc = evaluate(finetuned_model, test_loader, optimizer, criterion, device)

    scheduler.step()

    train_losses.append(train_loss)
    val_losses.append(val_loss)
    train_accuracies.append(train_acc)
    val_accuracies.append(val_acc)

    print(f"Train Loss: {train_loss:.4f}, Acc: {train_acc:.4f} | "
          f"Val Loss: {val_loss:.4f}, Acc: {val_acc:.4f}")

    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save(model.state_dict(), "best_finetuned_resnet18.pth")
        print("Saved best model.")

```



```

Epoch 1 / 5 (lr: 0.000010)
Train Loss: 0.9624, Acc: 0.7131 | Val Loss: 0.4794, Acc: 0.8569
Saved best model.

Epoch 2 / 5 (lr: 0.000010)
Train Loss: 0.3613, Acc: 0.8934 | Val Loss: 0.3147, Acc: 0.9051
Saved best model.

Epoch 3 / 5 (lr: 0.000009)
Train Loss: 0.2392, Acc: 0.9293 | Val Loss: 0.2616, Acc: 0.9172
Saved best model.

Epoch 4 / 5 (lr: 0.000008)
Train Loss: 0.1789, Acc: 0.9496 | Val Loss: 0.2350, Acc: 0.9257
Saved best model.

Epoch 5 / 5 (lr: 0.000007)
Train Loss: 0.1387, Acc: 0.9628 | Val Loss: 0.2220, Acc: 0.9282
Saved best model.

```

```

# Plots for Feature Extraction (Frozen Weights)
epochs_x = range(1, epochs + 1)
epochs_x_val = range(0, epochs + 1)

fig, (ax1, ax2) = plt.subplots(2, figsize=(8, 8))

ax1.plot(epochs_x, train_losses, label="Train Loss")
ax1.plot(epochs_x_val, val_losses, label="Validation Loss")
ax1.set_xlabel("Epoch")
ax1.set_ylabel("Loss")
ax1.set_title("Loss vs Epochs")
ax1.legend()
ax1.grid(alpha=0.3)

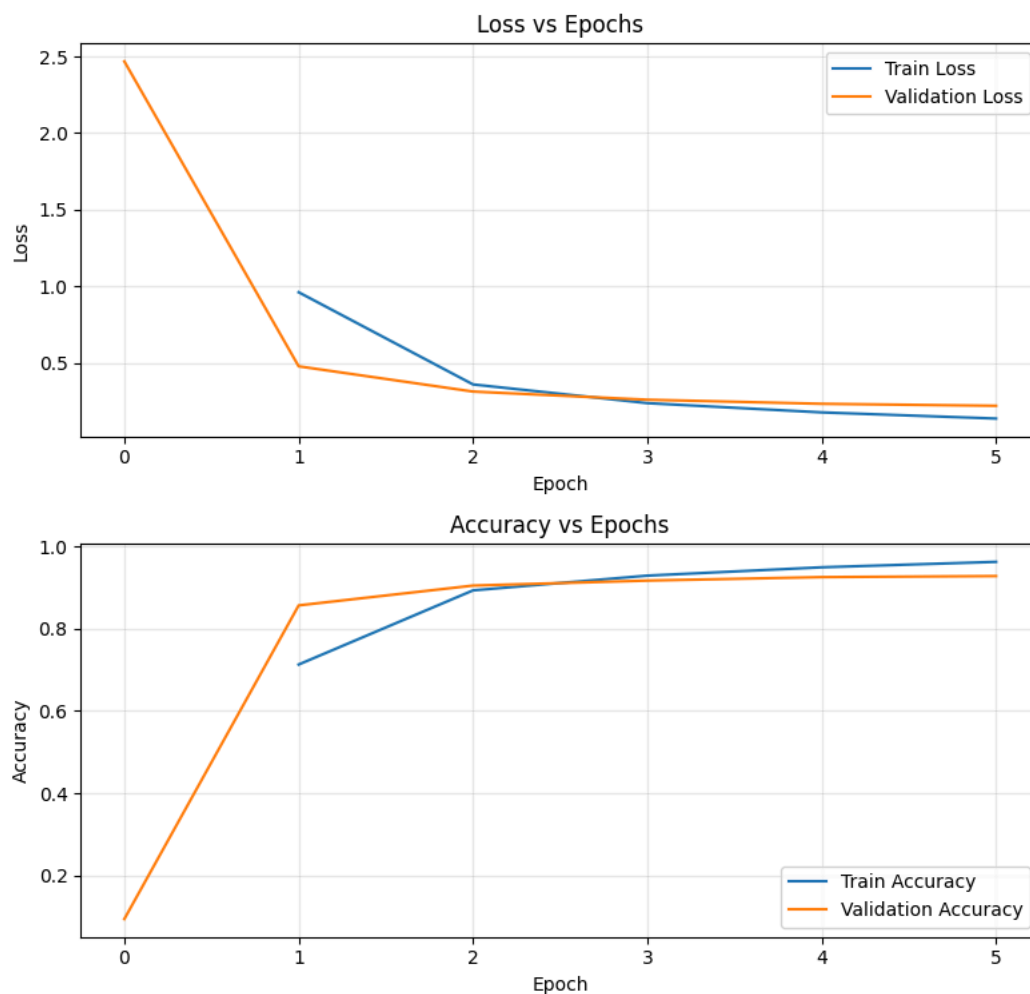
ax2.plot(epochs_x, train_accuracies, label="Train Accuracy")
ax2.plot(epochs_x_val, val_accuracies, label="Validation Accuracy")
ax2.set_xlabel("Epoch")
ax2.set_ylabel("Accuracy")
ax2.set_title("Accuracy vs Epochs")
ax2.legend()
ax2.grid(alpha=0.3)

fig.suptitle("Finetuning ResNet18", fontsize=16)
plt.tight_layout()
plt.show()

```



Finetuning ResNet18



✓ Training for Kaggle

```
class KaggleCIFAR10(nn.Module):
    def __init__(self, *args, **kwargs):
        super().__init__()
        self.base_model = models.resnet50(pretrained=True)

        num_features = self.base_model.fc.in_features
        self.base_model.fc = nn.Sequential(
            nn.Dropout(0.3),
            nn.Linear(num_features, 10)
        )

    def forward(self, x):
        return self.base_model(x)

kaggle_model = KaggleCIFAR10().to(device)

learning_rate = 1e-5
optimizer = torch.optim.Adam(kaggle_model.parameters(), lr=learning_rate, weight_decay=1e-4)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=10)
criterion = nn.CrossEntropyLoss()
print("Trainable parameters:", num_trainable_params(finetuned_model))

Trainable parameters: 11181642

kaggle_i_val_loss, kaggle_i_val_acc = evaluate(kaggle_model, test_loader, optimizer, criterion, device)

kaggle_train_losses, kaggle_val_losses = [], [kaggle_i_val_loss]
kaggle_train_accuracies, kaggle_val_accuracies = [], [kaggle_i_val_acc]
```



```

epochs = 10
counter = 0
kaggle_best_val_acc = 0
patience = 3

for epoch in range(epochs):
    print(f"\nEpoch {epoch + 1} / {epochs} (lr: {scheduler.get_last_lr()[0]:.6f})")

    kaggle_train_loss, kaggle_train_acc = train(kaggle_model, train_loader, optimizer, criterion, device)
    kaggle_val_loss, kaggle_val_acc = evaluate(kaggle_model, test_loader, optimizer, criterion, device)

    scheduler.step()

    kaggle_train_losses.append(kaggle_train_loss)
    kaggle_val_losses.append(kaggle_val_loss)
    kaggle_train_accuracies.append(kaggle_train_acc)
    kaggle_val_accuracies.append(kaggle_val_acc)

    print(f"Train Loss: {kaggle_train_loss:.4f}, Acc: {kaggle_train_acc:.4f} | "
          f"Val Loss: {kaggle_val_loss:.4f}, Acc: {kaggle_val_acc:.4f}")

    if kaggle_val_acc > kaggle_best_val_acc:
        kaggle_best_val_acc = kaggle_val_acc
        counter = 0
        torch.save({
            'epoch': epoch,
            'model_state_dict': kaggle_model.state_dict(),
            'optimizer_state_dict': optimizer.state_dict(),
            'scheduler_state_dict': scheduler.state_dict(),
            'val_acc': kaggle_val_acc,
        }, 'checkpoint_kaggle_resnet50.pth')
        print("Saved best model.")
    else:
        counter += 1

    if counter >= patience:
        print("Early Stop: Out of patience")
        break

```



Epoch 1 / 10 (lr: 0.000010)

Train Loss: 0.7880, Acc: 0.7829 | Val Loss: 0.2557, Acc: 0.9240

✓ Kaggle Submission

Epoch 2 / 10 (lr: 0.000010)

The following code is for you to make your submission to kaggle. Here are the steps you must follow:

1. Upload `cifar-test-data-sp25.npy` to the colab notebook by going to files on the left hand pane, then hitting "upload". This file may take roughly a minute to upload and you should not proceed to the following steps until it has completely finished uploading (students have run into issues where they were accidentally testing on a partially uploaded test set and getting garbage results).
2. Run the following cell to generate the dataset object for the test data. Feel free to modify the code to use the same transforms that you use for the training data. By default, this will reuse the transform variable.
3. In the second cell, write code to run predictions on the testing dataset and store them into an array called `predictions`.
4. Run the final cell which will convert your predictions array into a CSV for kaggle.
5. Go to the files pane again, and download the file called `submission.csv` by clicking the three dots and then download.

```
from PIL import Image
import os
```

```
class CIFAR10Test(torchvision.datasets.VisionDataset):
```

```
    def __init__(self, transform=None, target_transform=None):
        super(CIFAR10Test, self).__init__(None, transform=transform,
                                           target_transform=target_transform)
        assert os.path.exists("cifar10-test-data-sp25.npy"), "You must upload the test data to the file system."
        self.data = np.load("cifar10-test-data-sp25.npy", allow_pickle=False)
```

```
        self.data = np.vstack(self.data).reshape(-1, 3, 32, 32)
        self.data = self.data.transpose((0, 2, 3, 1)) # convert to HWC
```

```
    def __getitem__(self, index: int):
        img = self.data[index]
        img = Image.fromarray(img)
        if self.transform is not None:
            img = self.transform(img)
        return img
```

```
    def __len__(self) -> int:
        return len(self.data)
```

To save some hassle, we have provided the same transformation to be applied in the training data, but please # change this if you were to modify your training approach compared to what we have.

```
transform = transforms.Compose([
    transforms.Resize((224, 224)), # Resize to 224x224 (height x width)
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])
```

```
# Create the test dataset
testing_data = CIFAR10Test(
    transform=transform, # NOTE: Make sure transform is the same as used in the training dataset.
)
```

```
# Run trained model on test data
kaggle_test_dataloader = DataLoader(testing_data, batch_size=128, shuffle=False, num_workers=2)
predictions = []
```

```
checkpoint = torch.load("checkpoint_kaggle_resnet50.pth")
```

```
trained_model_kaggle = KaggleCIFAR10().to(device)
trained_model_kaggle.load_state_dict(checkpoint['model_state_dict'])
trained_model_kaggle.eval()
```

```
with torch.no_grad():
    for x in tqdm(kaggle_test_dataloader, desc="Running Kaggle Tests"):
```